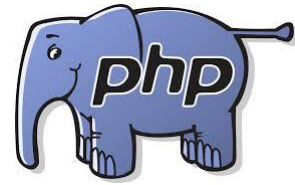


Unit 5

SERVER SIDE SCRIPTING(PHP)





Introduction

What is PHP?

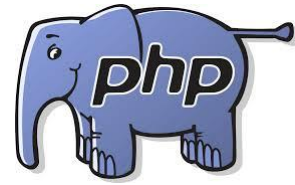
PHP (*Hypertext Preprocessor*) is a widely-used open source general-purpose server side scripting language that is especially suited for web development and can be embedded into HTML.



PHP's Place in the Web World

PHP is a programming language that's used mostly for building web sites. Instead of a PHP program running on a desktop computer for the use of one person, it typically runs on a web server and is accessed by lots of people using web browsers on their own computers. This section explains how PHP fits into the interaction between a web browser and a web server.

When you sit down at your computer and pull up a web page using a browser such as Internet Explorer or Mozilla, you cause a little conversation to happen over the Internet between your computer and another computer.



What's So Great About PHP?

PHP Is Free (as in Money)

You don't have to pay anyone to use PHP. There are no licensing fees, support fees, maintenance fees, upgrade fees, or any other kind of charge.

PHP Is Free (as in Speech)

As an open source project, PHP makes its innards available for anyone to inspect.

PHP Is Cross-Platform

You can use PHP with a web server computer that runs Windows, Mac OS X, Linux, Solaris, and many other versions of Unix.



Continue..

PHP Is Widely Used

As of March 2004, PHP is installed on more than 15 million different web sites.

PHP Hides Its Complexity

You can build powerful e-commerce engines in PHP that handle millions of customers.

PHP Is Built for Web Programming

Unlike most other programming languages, PHP was created from the ground up for generating web pages.



PHP Syntax

- ❑ PHP code starts with `<?php` and ends with `?>`
- ❑ Every PHP statements end with a semicolon(;
- ❑ PHP code save with `.php` extension
- ❑ PHP contain some HTML tag and PHP code
- ❑ You can place PHP code anywhere in your document



PHP Syntax

Start and End Tags

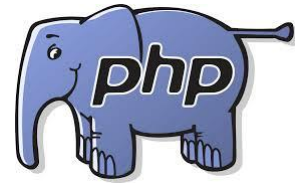
The above example uses `<?php` as the PHP start tag and `?>` as the PHP end tag. The PHP interpreter ignores anything outside of those tags. Text before the start tag or after the end tag is printed with no interference from the PHP interpreter.

```
<?php
```

```
//statements or blocks
```

```
?>
```

PHP can support multiple start and end tags. Each start tag must be closed by its own end tags.



Comments

Example: Single-line comments with // or #

```
// This line is a comment
```

```
print "Smoked Fish Soup ";
```

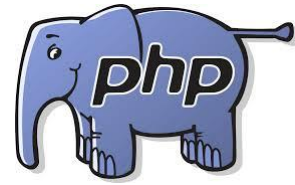
```
print 'costs $3.25.';
```

```
# Add another dish to the menu
```

```
print 'Duck with Pea Shoots ';
```

```
print 'costs $9.50.';
```

```
// You can put // or # inside single-line comments
```

Example: Multiline comments

`/* We're going to add a few things to the menu:`

- `- Smoked Fish Soup`
- `- Duck with Pea Shoots`
- `- Shark Fin Soup`

`*/`



Data Type

PHP has total of 8 data types which we used to construct our variables.

Integers are whole number without a decimal point.

Doubles are floating point numbers like 3.1415, 2.50

Boolean have only two possible value either true or false.

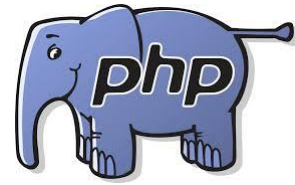
NULL is special type that only has one value: NULL

Strings are sequence of characters like php supports string operations

Arrays are named and indexed collection of other values.

Object are instance of program defined classes which can package up both other kinds of values and functions that are specified to the class.

Resources are special variables that hold references to resources external to php (such as database connection).



String Example

```
<?php  
print 'I would like a bowl of soup.';  
print 'chicken';  
print '06520';  
print '"I am eating dinner," he growled.';  
?>
```

Output:

I would like a bowl of soup.chicken06520"I am eating dinner," he growled.



The output of those four print statements appears all on one line. No line breaks are added by print. You may also use echo used in some PHP programs to print text. It works just like print. The single quotes aren't part of the string. They are delimiters, which tell the PHP interpreter where the start and end of the string is.



If you want to include a single quote inside a string surrounded with single quotes, put a backslash (\) before the single quote inside the string:

```
<?php
```

```
print 'We\'ll each have a bowl of soup.';
```

```
?>
```

Output:

We'll each have a bowl of soup.



The backslash tells the PHP interpreter to treat the following character as a literal single quote instead of the single quote that means "end of string." This is called escaping, and the backslash is called the escape character. An escape character tells the system to do something special with the character that comes after it. Inside a single-quoted string, a single quote usually means "end of string." Preceding the single quote with a backslash changes its meaning to a literal single quote character.

You can also delimit strings with double quotes. Double-quoted strings are similar to single-quoted strings, but they have more special characters.

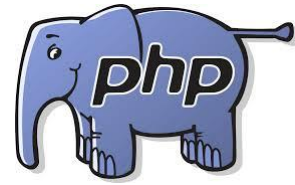


Special characters in double-quoted strings	
Character	Meaning
<code>\n</code>	Newline(ASCII 10)
<code>\r</code>	Carriage return (ASCII 13)
<code>\t</code>	Tab (ASCII 9)
<code>\\</code>	<code>\</code>
<code>\\$</code>	<code>\$</code>
<code>\"</code>	<code>"</code>
<code>\0..\777</code>	Octal (Base 8) Number
<code>\x0..\xFF</code>	Hexadecimal (Base 16) Number



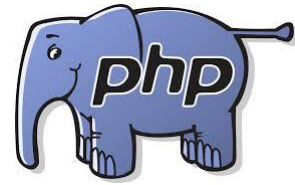
The biggest difference between single-quoted and double-quoted strings is that when you include variable names inside a double-quoted string, the value of the variable is substituted into the string, which doesn't happen with single-quoted strings.

For example, if the variable `$user` held the value `Bill`, then `'Hi $user'` is just that: `Hi $user`. However, `"Hi $user"` is `Hi Bill`.



Constant

A constant is a name or an identifier for a simple value. A constant value cannot change during the execution of the script. By default a constant is a case-sensitive. By convention, constant identifiers are always uppercase.



Constant() function

As indicated by name, this function will return the value of the constant.

Example:

```
<?php  
define ("MINSIZE",50);  
echo MINSIZE;  
echo constant ("MINSIZE"); //same thing as above  
?>
```

Output:

?



Numbers

Numbers in PHP are expressed using familiar notation, although you can't use commas or any other characters to group thousands. You don't have to do anything special to use a number with a decimal part as compared to an integer.



Example: Numbers

```
print 56;
```

```
print 56.3;
```

```
print 56.30;
```

```
print 0.774422;
```

```
print 16777.216;
```

```
print 0;
```

```
print -213;
```

```
print 1298317;
```

```
print -9912111;
```

```
print -12.52222;
```

```
print 0.00;
```



Using Different Kinds of Numbers

Internally, the PHP interpreter makes a distinction between numbers with a decimal part and those without one. The former are called **floating-point** numbers and the latter are called **integers**.

Floating-point numbers take their name from the fact that the decimal point can "float" around to represent different amounts of precision.

Boolean



Boolean data types are used in conditional testing. Hold only two values, either TRUE(1) or FALSE(0).

Successful events will return true and unsuccessful events return false. NULL type values are also treated as false in Boolean.

Apart from NULL, 0 is also considered false in boolean. If a string is empty then it is also considered false in boolean data type.

```
<?php
```

```
if(TRUE)
```

```
    echo "This condition is TRUE";
```

```
if(FALSE)
```

```
    echo "This condition is not TRUE";
```

```
?>
```

Array



Array is a compound data type that can store multiple values of the same data type. Below is an example of an array of integers. It combines a series of data that are related together.

Indexed array:

```
<?php  
$numarr = array( 10, 20 , 30);  
echo "First Element: $numarr[0]\n";  
echo "Second Element: $numarr[1]\n";  
echo "Third Element: $numarr[2]\n";  
?>
```

Array



Associative array: Associative arrays are arrays that use named keys that you assign to them.

```
<?php
```

```
$age = array("Aman"=>"21","Binita"=>"19","Chandan"=>"21");
```

```
// or you can declare array element by $age['Binita']="19";
```

```
echo "Binita is ". $age['Binita']. "years old"
```

```
?>
```

Foreach loop for displaying elements of associative array with key and value.

```
foreach($age as $x=>$x_value)
```

```
{
```

```
echo "Key=".$x.", Value=".$x_value;
```

```
echo "<br>"
```

```
}
```

```
?>
```


Object

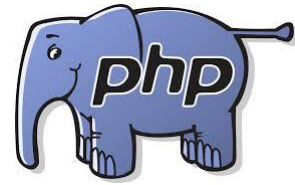


Objects

Objects are defined as instances of user-defined classes that can hold both values and functions and information for data processing specific to the class. This is an advanced topic and will be discussed in detail in further articles. When the objects are created, they inherit all the properties and behaviors from the class, having different values for all the properties.

Objects are explicitly declared and created from the new keyword.

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;
    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}
$obj=new Fruit();
$obj.set_name('Grape');
$obj.get_name();
?>
```



Variables

Variables hold the data that your program manipulates while it runs, such as information about a user that you've loaded from a database or entries that have been typed into an HTML form. In PHP, variables are denoted by \$ followed by the variable's name. To assign a value to a variable, use an equals sign (=). This is known as the assignment operator.



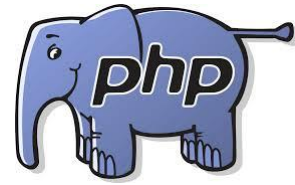
Examples:

```
$plates = 5;
```

```
$dinner = 'Beef Chow-Fun';
```

```
$cost_of_dinner = 8.95;
```

```
$cost_of_lunch = $cost_of_dinner;
```



Acceptable variable names

`$size`

`$drinkSize`

`$my_drink_size`

`$_drinks`

`$drink4you2`



Unacceptable variable names

Variable name

Flaw

\$2hot4u
number

Begins with a

\$drink-size

Unacceptable character: -

\$drinkmaster@example.com

Unacceptable characters: @ and .

\$drink!lots

Unacceptable character: !

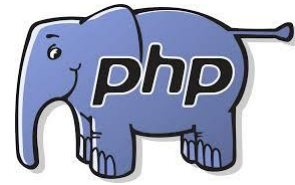
\$drink+dinner

Unacceptable character: +



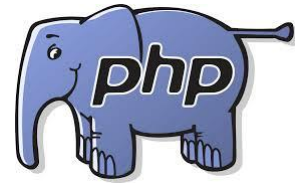
Variable names

Variable names are case-sensitive. This means that variables named `$dinner`, `$Dinner`, and `$DINNER` are separate and distinct.



Operating on Variables

Arithmetic and string operators work on variables containing numbers or strings just like they do on literal numbers or strings.



Example: Operating on variables

```
<?php
$price = 3.95;
$tax_rate = 0.08;
$tax_amount = $price * $tax_rate;
$total_cost = $price + $tax_amount;
$username = 'james';
$domain = '@example.com';
$email_address = $username . $domain;
print 'The tax is ' . $tax_amount;
print "\n"; // this prints a linebreak
print 'The total cost is ' . $total_cost;
print "\n"; // this prints a linebreak
print $email_address;
?>
```

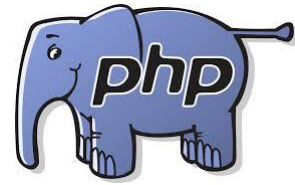



Output:

The tax is 0.316

The total cost is 4.266

james@example.com



Example: Combined assignment and addition

// Add 3 the regular way

```
$price = $price + 3;
```

// Add 3 with the combined operator

```
$price += 3;
```

Output:

?



Example: Combined assignment and concatenation

```
$username = 'james';
```

```
$domain = '@example.com';
```

```
// Concatenate $domain to the end of $username the regular way
```

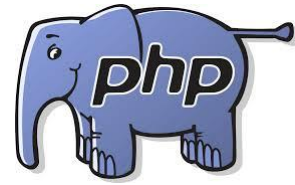
```
$username = $username . $domain;
```

```
// Concatenate with the combined operator
```

```
$username .= $domain;
```

Output:

?



Example: Incrementing and decrementing

```
// Add one to $birthday
$birthday = $birthday + 1;

// Add another one to $birthday
++$birthday;

// Subtract 1 from $years_left
$years_left = $years_left - 1;

// Subtract another 1 from $years_left
--$years_left;
```

Output:

?



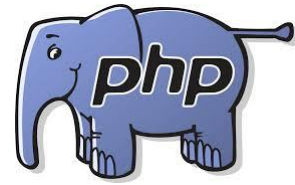
Putting Variables Inside Strings

Example: Variable interpolation

```
<?php  
$email = 'jacob@example.com';  
print "Send replies to: $email";  
?>
```

Output:

Send replies to: jacob@example.com



define() function

To define a constant, we have to use `define()` function and to retrieve the value of a constant we have to simply specify its name.



Arithmetic Operators

Some basic operations between numbers are shown in example below.



Example: Math operations

```
print 2 + 2;  
print 17 - 3.5;  
print 10 / 3;  
print 6 * 9;  
print 17 % 3;
```

Output:

```
4  
13.5  
3.33333333333333  
54  
2
```




Operator:

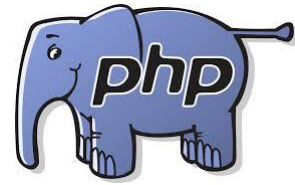
Arithmetic Operator

Comparison Operator

Logical or Relational Operator

Assignment Operator

Conditional or ternary Operator



Arithmetic Operator

Assume variable A holds 10 and variable B holds 20 then.

Operator	Description	Example
+	Adds two operands from the first	A+B will give 30
-	Subtract second operand from the first	A-B will give -10
*	Multiply both operands	A*B will give 200
/	Divide the numerator by denominator	B/A will give 2
%	Modulus operator & remainder after an integer division	B%A will give 0
++	Increment operator increases integer value by one	A++ will give 11
--	Decrement operator decreases integer value by one	A-- will give 9



Comparison Operator

Assume variable A holds 10 and variable B holds 20 then.

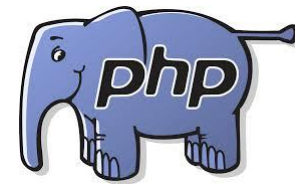
Operator	Description	Example
==	Checks if the values of two operands are equal or not if yes then condition becomes true.	(A==B) is not true
!=	Checks if the values of two operands are equals or not, if not equal the condition becomes true.	(A!=B) is true
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A>B) is not true
<	Check if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A<B) is true
>=	Check if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A>=B) is not true
<=	Check if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A<=B) is true



Logical Operator

Assume variable A holds 10 and variable B holds 20 then.

Operator	Description	Example
AND	It is called logical AND operator. If both the operands are true the condition becomes true.	(A AND B) is true
OR	It is called logical OR operator. If any of the two operands are non-zero then condition becomes true.	(A OR B) is true
&&	It is called logical AND operator. If the both the operands are non zero then the condition becomes true.	(A && B) is true
	It is called logical OR operator. If any of the two operands are non-zero then condition becomes true.	(A B) is true
!	It is called logical NOT operator, use to reverse the logical state of its operand. If a condition is true then logical NOT operator will make false.	!(A && B) is false



Assignment Operator

Operator	Description	Example
=	Simple assignment operator, assign values from right side operands to left side operands.	C=A+B will assign to value of A+B into C.
+=	Add and assignment operator, adds right operand to the left operand and assign the result to left operand.	C+=A is equivalent to C=C+A
-=	Subtract and assignment operator, subtract right operand from the left operand and assign the result to left operand	C-=A is equivalent to C=C-A
=	Multiply and assignment operator, multiply right operand from the left operand and assign the result to left operand	C=A is equivalent to C=C*A
/=	Divide and assignment operator, divide right operand from the left operand and assign the result to left operand	C/=A is equivalent to C=C/A
%=	Modulus and assignment operator, modulus right operand from the left operand and assign the result to left operand	C%=A is equivalent to C=C%A



Control Structure

If statement

If else statement

Switch statement

***Same as javascript



Loops

- ❑ For Loop

- ❑ While loop

- ❑ Do while loop

**same as javascript

- ❑ **Foreach loop: example mentioned in previous slide (array)**

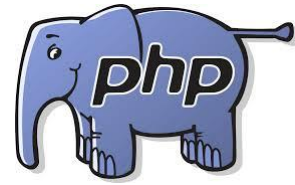


PHP forms

Forms are essential components of a website used register a new account, sign in or login to the account. Using a form in a PHP program is a two step activity. Step one is to display the form which involves constructing HTML tags and UI's. Step two is processing the submitted for information submitted by the user.

Accessing Form Elements:

The PHP superglobals `$_GET` and `$_POST` are used to collect form data. `$_GET` is an array of variables passed to the current script via the URL parameters. `$_POST` is an array of variables passed to the current script via the HTTP POST method



\$_GET method

Information sent from a form with the GET method is visible to everyone(all variable names and values are displayed in the URL).

GET also has limits on the information to send. The limitation is about 2000 characters. GET may be used for sending non-sensitive data. GET should never be used for sending passwords or other sensitive information.



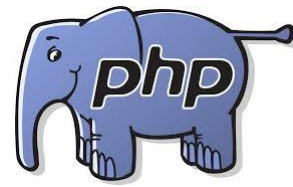
\$_POST method

Information sent from a form with the POST method is invisible to others(all names/values are embedded within the body of the HTTP request) and has no limits on the amount of information to be sent. POST may be used for sending sensitive data. Developers prefer POST for sending form data.

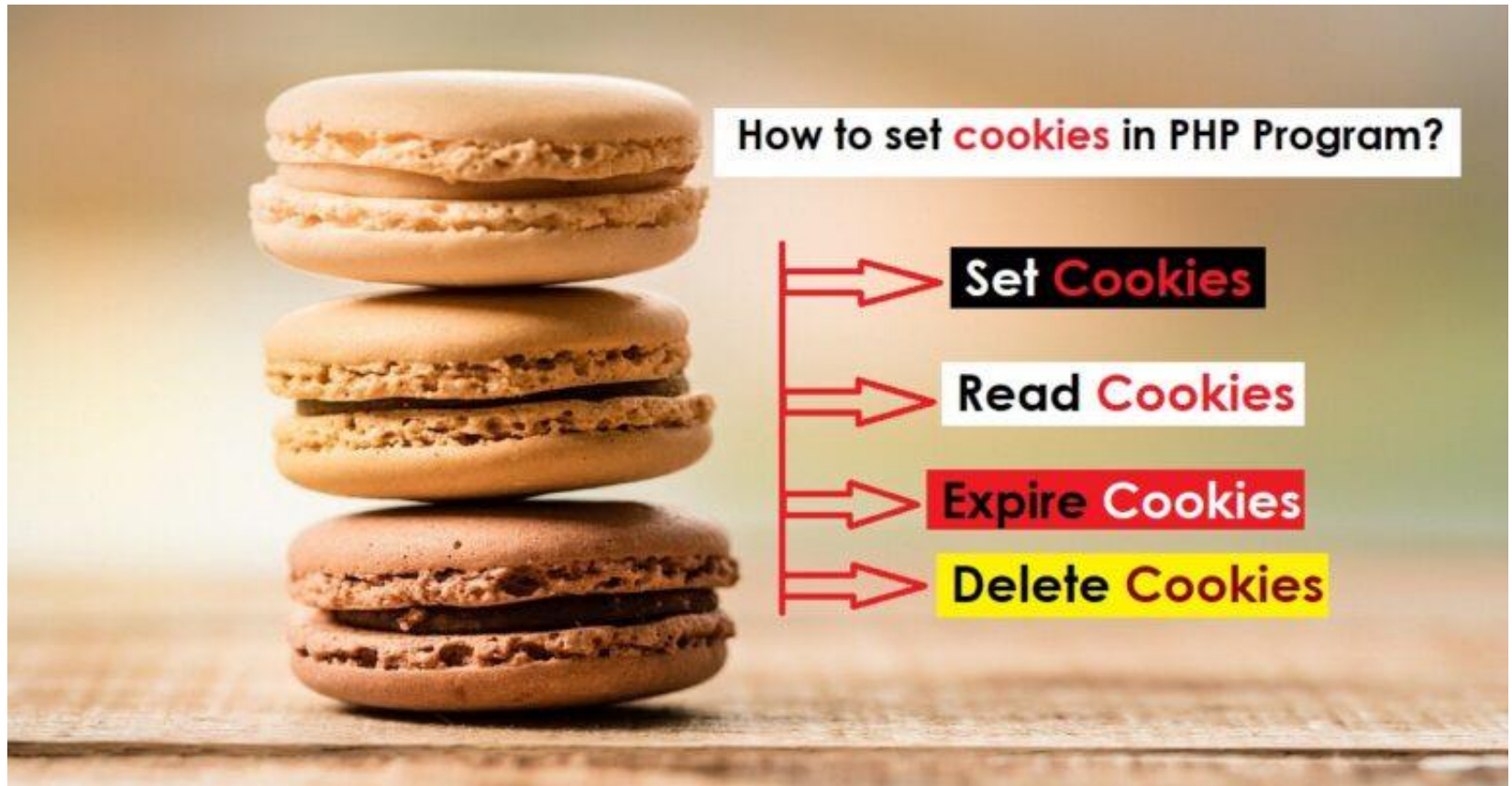


Form validation

Similar to Javascript.
Study yourself



Cookies





Cookies

Cookie: A cookie is variable that is stored on the visitors computer. Each time the same computer requests a page with a browser it will sent the cookie too. With PHP, we can both create and retrieve cookie values.

*Creating cookies with PHP

Syntax: `setcookie(name,value,expire,path,domain,...);`

Only name parameter is compulsory. And other are optional.

To delete a cookie, we use the `setcookie()`function with an expiration date in the past.



Cookies

Example: Creating and retrieve a cookie

```
<?
```

```
$cookie_name="username";
```

```
$cookie_value="John Doe";
```

```
setcookie($cookie_name,$cookie_value,time()+(86400*30),"/");
```

```
If(!isset($_COOKIE[$cookie_name])){
```

- echo \$cookie_name."is not set";
- }

```
else{
```

- echo \$cookie_name."is set ";
- }
- ?>



Session

A session is a way to store information(in variables) to be used across multiple pages. Unlike a cookie, the information is not stored on the users computer rather session is stored in server. Session variables hold information about one single user and are available to all pages in one application.

Why do we need session?

- When we work with an application, we open it, do some changes, and then we close it. This is much like session. The computer knows who we are. It knows when we start the application and when we end. But on the internet there is one problem: the web server does not know who we are or what er do, because HTTP address doesn't maintain state. Session variable solve this problem by storing user information to be used across multiple pages. By, default, session variables last until the user closes the browser.



Session

Uses of Session:

- ☐ Session can help to uniquely identify each client from another
- ☐ Session is secure and transparent from the user
- ☐ It is easy to implement and we can store any kind of object
- ☐ It helps to maintain user state and data all over the application
- ☐ It stores client data separately

Starting PHP Session:

A PHP session is easily started by making a call to the `session_start()` function. Session variables are stored in associative array called `$_SESSION[]`. We make use of `isset()` function to check if session variable is already set or not.



Session

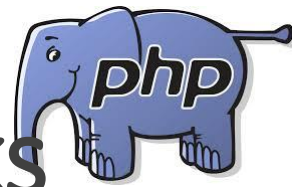
Example: Starting session and setting session values

```
<?php
//start the session
session_start();
//set session variables
$_SESSION["username"]="john";
$_SESSION["email"]="lucky@gmail.com";
echo "Session variables are set";
?>
```

****** session_unset() or session_destroy() is used to remove all global variables and destroy session.



<u>Cookies</u>	<u>Sessions</u>
Cookies are stored on the user's computer.	Sessions are stored on the server.
A cookie can store a limited amount of data, a maximum of 4KB.	The session can store an unlimited amount of data.
The cookie does not depend on the Session.	The session depends on the cookie.
The cookie expires according to the time of expiry set for it	The session expires after the user closes the web browser.
Cookie has many security issues as it can be accessed by anyone easily.	The session is secure as it cannot be accessed by anyone easily.
There is no function available to disable a Cookie.	A Session can be disabled by using the <u>session_destroy()</u> function.
The <u>setcookie()</u> function should always be used prior to tag.	The <u>session_start()</u> function should always appear prior to tag.



Introduction to PHP Frameworks

1.CodeIgniter:



CodeIgniter is a powerful PHP framework with a very small footprint, built for developers who need a simple and elegant toolkit to create full featured web applications. It works on MVC(Model View Controller) framework

Installation steps in book/uses



2. Laravel: Laravel is a web application framework with expressive, elegant syntax. Laravel attempts to take the pain out of development by easing common tasks used in the majority of web projects such as authentication, routing, sessions and caching. Laravel is a powerful MVC PHP framework.

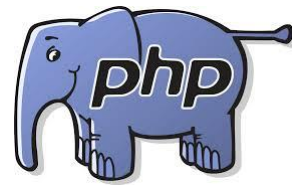
Installation steps in book/uses



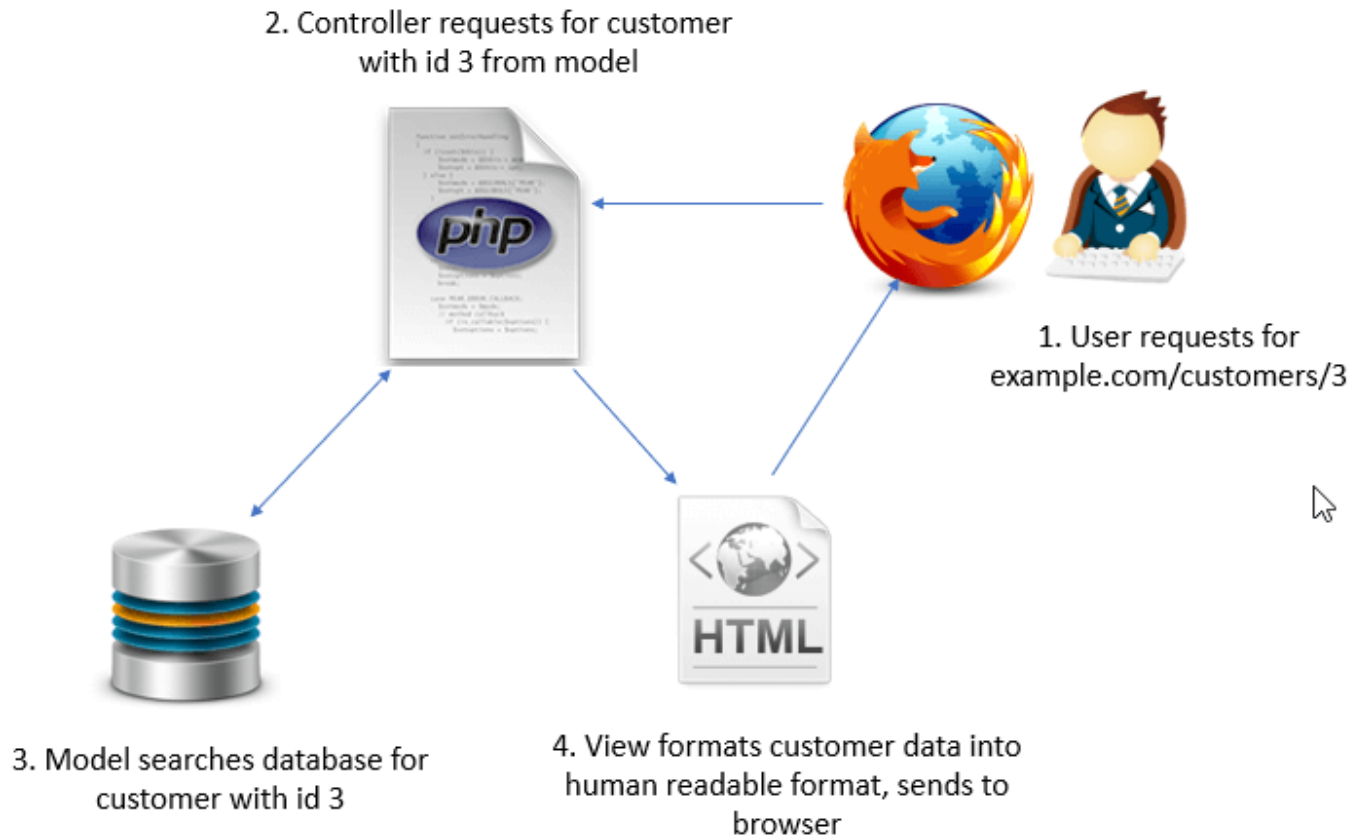
3.Wordpress:

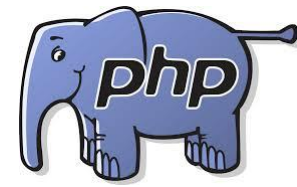
WORDPRESS

Wordpress is a free and open source content management system.(CMS) based on PHP and MYSQL. It is the most widely used CMS software in the world. Wordpress is an industry leading website and blog building tool. There is an entire community of Wordpress users and developers who are able to assist one another. User have countless options for free and paid plugins and site templates for their use in creating their site.

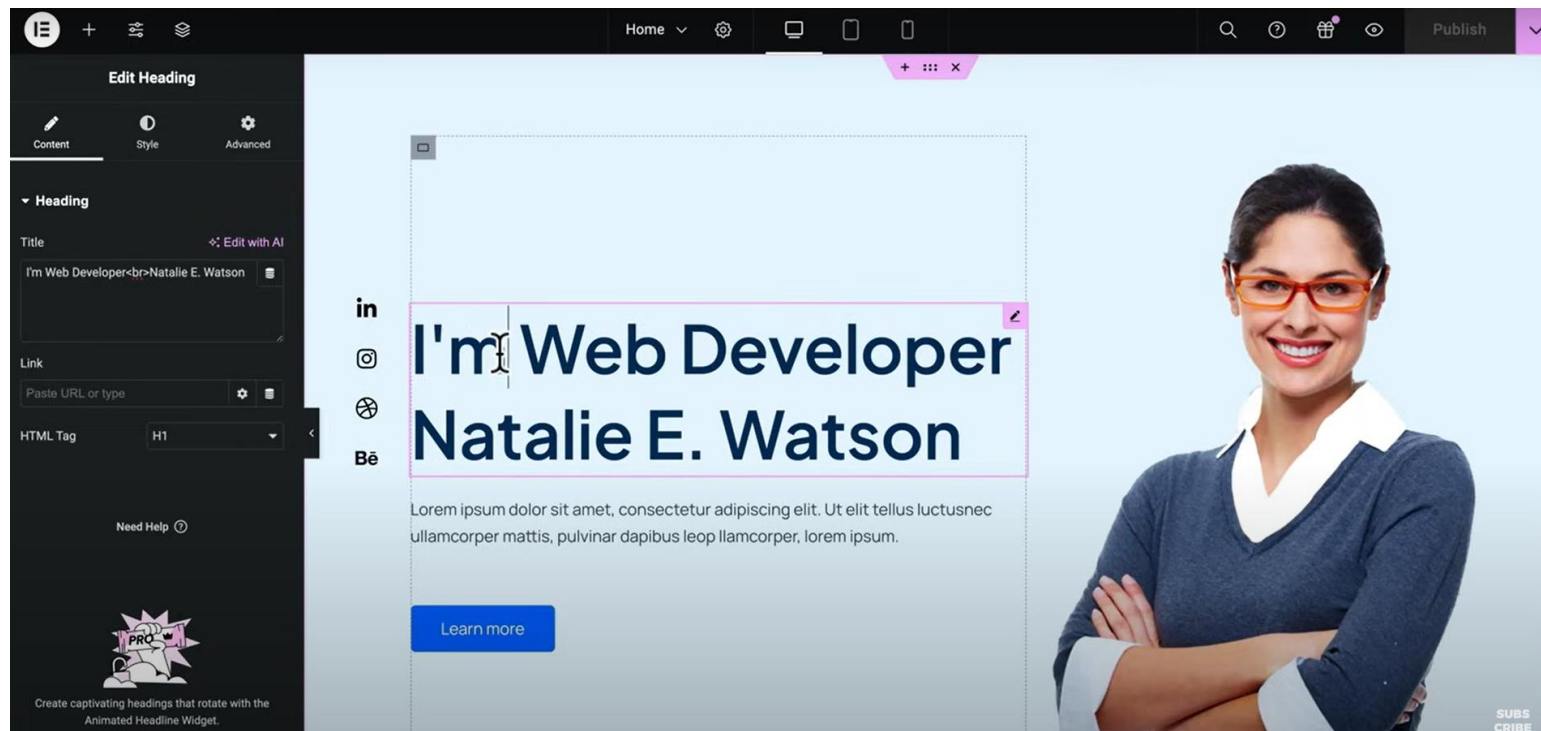


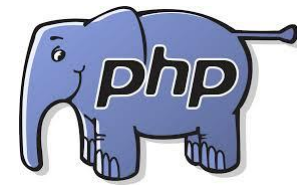
MVC architecture





WordPress interface





WordPress interface

The screenshot displays the WordPress Gutenberg editor interface. On the left is a dark sidebar with the 'Edit Button' panel active, showing tabs for 'Content', 'Style', and 'Advanced'. The 'Content' tab is selected, showing fields for 'Type' (Default), 'Text' (Start here), 'Link' (https://meticsmedia11.com/portfolio/), 'Icon', and 'Button ID'. A note at the bottom of the sidebar states: 'Please make sure the ID is unique and not used elsewhere on the page this form is displayed. This field allows A-z 0-9 & underscore chars without spaces.' Below this is a 'Need Help' link. The main editing area shows a light blue background with a woman's image on the right. The text 'I'm a Youtuber :)' is prominently displayed. To the left of the text are social media icons for YouTube, Instagram, and Facebook. Below the text is a blue button labeled 'Start here'. A mouse cursor is hovering over the button. The top of the editor shows a navigation bar with 'Home', 'Settings', 'Preview', and 'Publish' buttons. The bottom of the image has a green bar with the text 'SUBSCRIBE'.



Assignment

1. Define Class and Object in PHP
2. Define Events in PHP
3. Define PHP Superglobals.
\$_REQUEST
\$_SERVER